Goldsmiths
UNIVERSITY OF LONDON

Linky:
Mini Project
(Public Version)

Author: Zhao Wang

DATABASES AND THE WEB

2023

Contents

1 Outline **1**

2 Architecture **2**

2.1 Diagram 2

2.2 Description 3

3 Data Model **4**

3.1 Diagram 4

3.2 Description 4

3.2.1 Users 4

3.2.2 Access Tokens (tokens) 5

3.2.3 URL Entries (links) 5

4 User Functionality **6**

4.1 Link Management 6

4.2 Public Links 7

4.3 Search Links 7

4.4 Access Control 8

5 Security **9**

5.1 SQL Injection 9

5.2 Malicious User Content 9

5.3 Denial of Service Attacks 9

5.4 Brute-Force Attacks 10

5.5 Reverse Proxy Bypass & CNAME Pranks 10

5.6 Other Unknown Attacks 10

6 API Usage **11**

6.1 Authentication 11

6.2 Endpoints 12

6.2.1 List Public Links 12

6.2.2 List Private Links 12

6.2.3 List All Accessible Links 12

6.2.4 Shorten a URL into a Link 13

6.2.5 Get Link Detail 13

6.2.6 Delete a Link 14

6.2.7 Make a Link Public 14

6.2.8 Make a Link Private 14

6.2.9 Ping 15

7 Advanced Techniques **16**

7.1 Containerization 16

7.2 Background Tasks 16

7.3 Reverse Proxy 16

7.4 Markdown Documentation Module from Scratch 16

7.5 LaTeX 17

7.6 Shell Scripting 17

References **18**

Chapter 1

Outline

Linky is a simple and fast URL shortener service. Visitors need to register an account in order to add, modify or delete their own URL entries, and all entries will be checked against VirusTotal[1]'s free API for potential malicious content. Entries can then be made public for anyone to access.

There is a search page as well. A user can search public entries or this user's own entries. Public entries tagged as malicious cannot be searched.

There is also REST API. These endpoints use the "Authorization" HTTP header, and a user access token can be obtained with the user's username and password.

Linky uses Express.js as the web server, and MySQL as its main database. Background tasks such as virus check status updates are queued using Redis[2]. The whole project is easily deployable using latest Docker[3] Compose, and currently I am using Caddy[4] with hardened configuration as its reverse proxy.

A production deployment is on Hetzner[5] under my own domain:

https://miniproject.gscoursework.thealois.com/

Credentials: (Latest credentials are also available in the code repository READMEs)

- username (three 'b's): `bbb`
- password (seven 'a's): `aaaaaaa`

The code repositories are available:

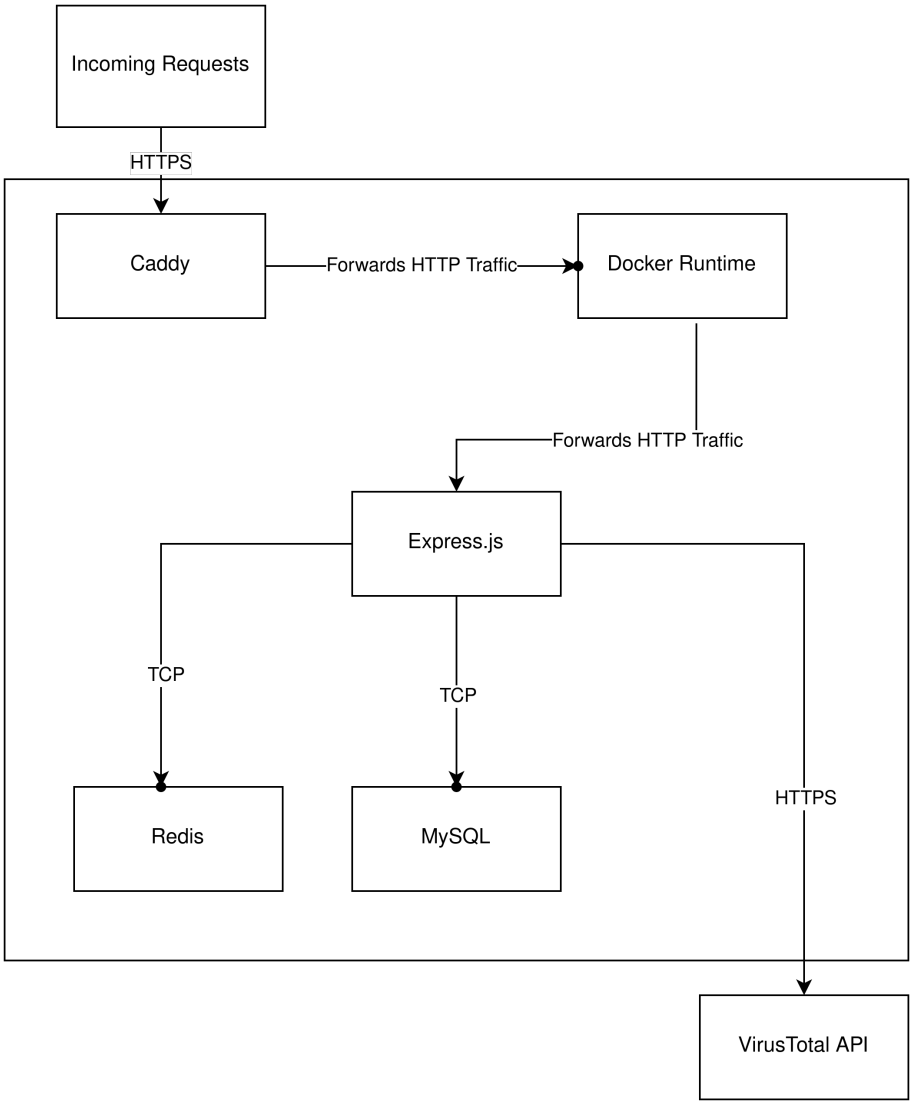
GitHub: https://github.com/iatyl/db-and-the-web

Goldsmiths GitLab: https://gitlab.doc.gold.ac.uk/zwang038/db-and-the-web

Chapter 2

Architecture

2.1 Diagram



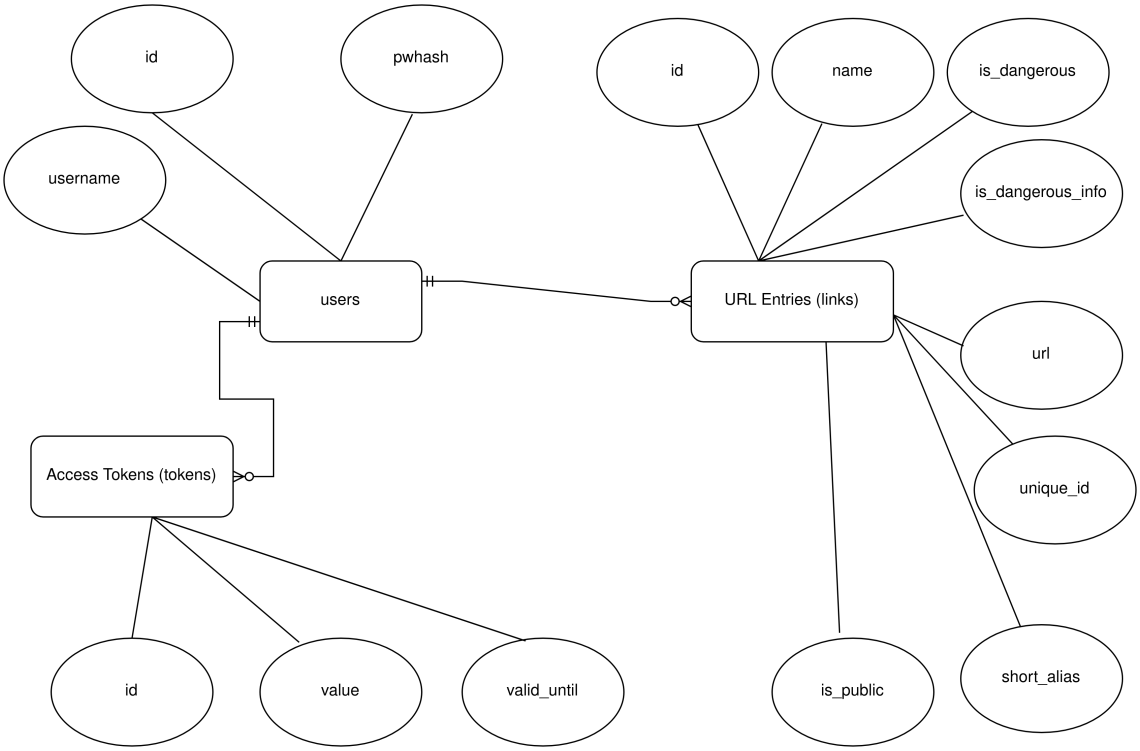
2.2 Description

This architecture uses Caddy[4] to receive HTTPS requests. The Express.js server always receives plain HTTP requests, decrypted by Caddy[4] and forwarded by Caddy[4] and Docker[3]'s host-container network. MySQL is used to store user data, access tokens and URL entries. Redis[2] is used by the bee-queue[6] package, to queue virus check requests so they can be updated in the background. The Express.js server will use a free API key to request URL virus checks from VirusTotal[1] using HTTPS.

Chapter 3

Data Model

3.1 Diagram



3.2 Description

3.2.1 Users

'`pwhash`' is the user password hash.

3.2.2 Access Tokens (tokens)

This is for REST API user authentication. '`valid_until`' is a UNIX timestamp, after which the token will be invalidated. The 'value' field is the actual token string.

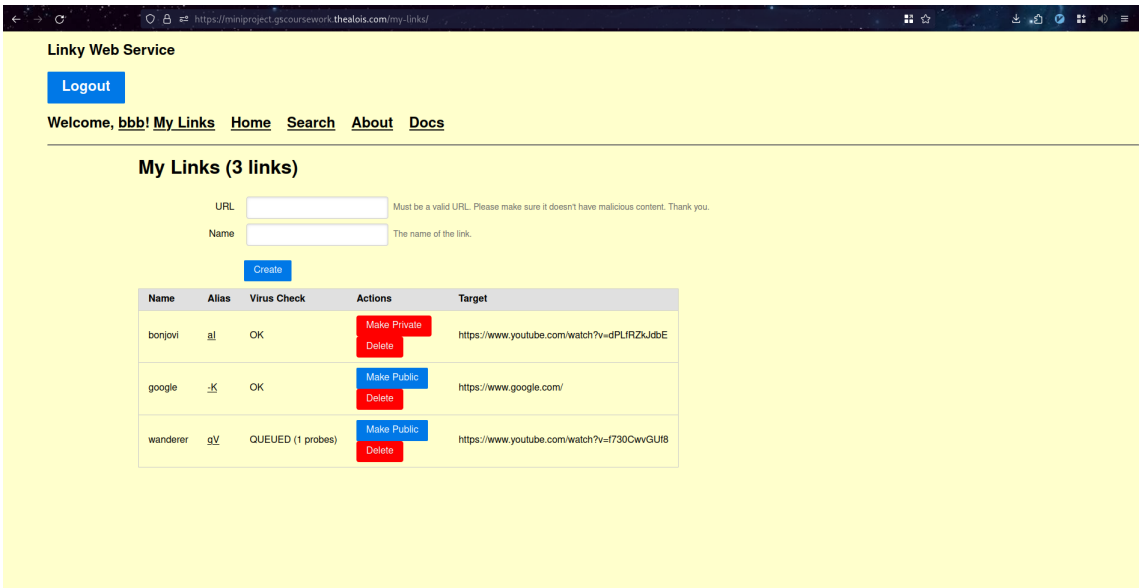
3.2.3 URL Entries (links)

The '`unique_id`'s are UUIDv4 strings and help the background tasks identify URL entry objects before the data is inserted into the database. '`is_dangerous_info`' is used to track the virus check progress and display more detailed results when the virus checks are done. '`short_alias`' is used in shortened URLs. The functionalities of other fields are obvious.

Chapter 4

User Functionality

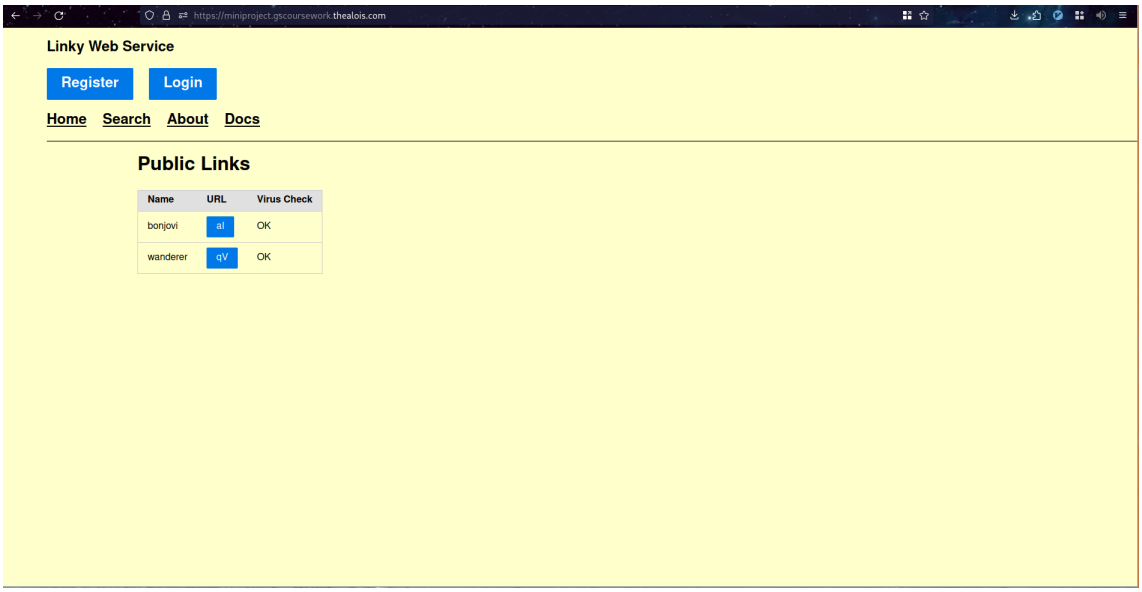
4.1 Link Management



After logging in, the user will be redirected to "My Links" page where the user is able to create, delete or change the public accessibility of their own links.

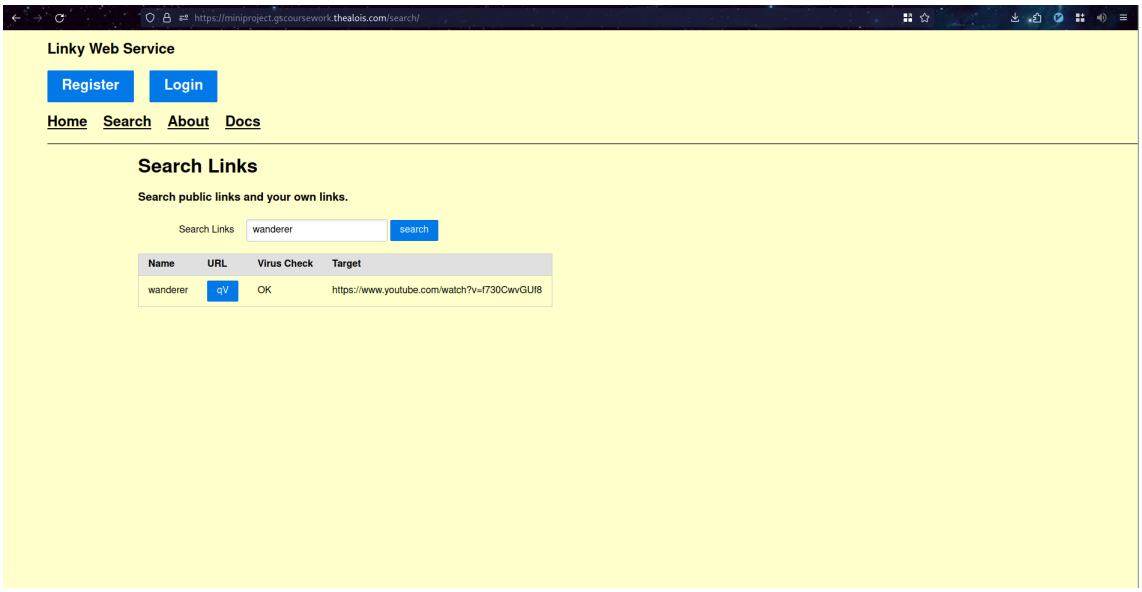
The "`QUEUED (1 probes)`" text means that the virus check request is queued in local Redis[2] as well as at VirusTotal[1], and "`(1 probes)`" means that the background task process has asked VirusTotal[1] for status update for exactly once. After the virus check is finished, this text will be updated to "OK" or, if it's tagged as malicious, the URL's virus info.

4.2 Public Links



The home page of Linky shows every publicized short URL, unless it has been tagged as malicious or dangerous.

4.3 Search Links

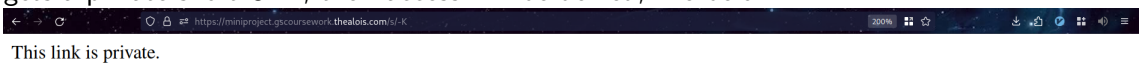


Our visitors are able to search for shortened links by parts of their names or URLs. Once the user is logged in, the user will also get results from private links.

4.4 Access Control

All dangerous URLs are blocked, like below:

All private links are only accessible to their owners. If any other user or unauthenticated visitor gets a private short URL, their access will be denied, like below:



Chapter 5

Security

5.1 SQL Injection

All user input are escaped by the "mysql" npm package, making sure all SQL statements will run as expected.

5.2 Malicious User Content

This issue is address by making virus check mandatory. The project initiates a virus check at VirusTotal[1] before any user shortens a URL. During the virus check-up, all queued URLs are assumed to be safe, but once a URL is tagged as malicious, the redirection is blocked.

5.3 Denial of Service Attacks

This section concerns the Express.js web server, and will not talk about the DoS attacks on the physical server. The Linky project is structured to have a reverse proxy in front of Express. All DoS attempts that are able to go through Caddy[4] are assumed to cause Express to generate some error log. My specific practice is to have "fail2ban"[7] to monitor and parse STDERR logs printed by Express, and block suspicious IP addresses.

5.4 Brute-Force Attacks

This kind of attacks are similar to DoS attacks but are mainly about user authentication pages, and thus are easier to identify. I use Caddy[4] to ratelimit the requests to the login page and the register page. Detailed configuration are available in my code repository's '`./config/Caddyfile`' file.

5.5 Reverse Proxy Bypass & CNAME Pranks

I make all the docker containers only expose their ports to localhost (127.0.0.1), so it's impossible to bypass the reverse proxy, which provides all kinds of protection, to access the Express server directly on port 8000. This will also deal with CNAME DNS record pranks, since if the 'HOST' header mismatches, Caddy[4] simply will not serve Linky's content.

5.6 Other Unknown Attacks

There could also be zero-day vulnerabilities on Caddy[4] or Express. My Caddy[4] config (`./config/Caddyfile`) strips the "Server" and "x-powered-by" headers to hide my technology stack to minimize the attack surface.

Chapter 6

API Usage

All examples here use the "curl"[8] program, which is available on all UNIX operating systems and Windows Subsystem for Linux, like below:

```
curl -H 'Content-Type: application/json' \
     -H 'Authorization: Token <TOKEN>' \
     -X <METHOD> -d '<JSON>' <URL>
```

The **API Usage** chapter is also available in markdown:

https://github.com/iatyl/db-and-the-web/blob/main/misc/api_docs.md

6.1 Authentication

The API endpoints authenticate its users via the access token retrieved from the "Authentication" header.

Here is how to get an access token (valid for 72 hours):

```
curl -H 'Content-Type: application/json' \
     -X POST \
     -d '{"username":"bbb","password":"aaaaaaa"}' \
     https://miniproject.gscoursework.thealois.com/api/auth/get-token/
```

Sample response (here the access token is in the "data" field):

```
{"data":"588d087c-58d1-4129-b610-d13a70309cee","err":null}
```

6.2 Endpoints

6.2.1 List Public Links

Sample request:

```
curl -H 'Content-Type: application/json' \
     -H 'Authorization: Token 588d087c-58d1-4129-b610-d13a70309cee' \
     -X GET \
     https://miniproject.gscoursework.thealois.com/api/links/list-public/
```

Sample response:


```
{"data":[{"name":"bonjovi","alias":"aI","target":"https://www.youtube.com/watch
    ?v=dPLfRZkJdbE","is_dangerous":false,"virus_info":"OK","is_public":true}],"
    err":null}
```


6.2.2 List Private Links

Sample request:

```
curl -H 'Content-Type: application/json' \
     -H 'Authorization: Token 588d087c-58d1-4129-b610-d13a70309cee' \
     -X GET \
     https://miniproject.gscoursework.thealois.com/api/links/list-private/
```

Sample response:


```
{"data":[{"name":"bonjovi","alias":"aI","target":"https://www.youtube.com/watch
    ?v=dPLfRZkJdbE","is_dangerous":false,"virus_info":"OK","is_public":true},{"
    name":"google","alias":"-K","target":"https://www.google.com/","
    is_dangerous":false,"virus_info":"OK","is_public":false},{"name":"wanderer
    ","alias":"qV","target":"https://www.youtube.com/watch?v=f730CwvGUf8","
    is_dangerous":false,"virus_info":"OK","is_public":false}],"err":null}
```


6.2.3 List All Accessible Links

Sample request:

```
curl -H 'Content-Type: application/json' \
     -H 'Authorization: Token 588d087c-58d1-4129-b610-d13a70309cee' \
     -X GET \
     https://miniproject.gscoursework.thealois.com/api/links/list-accessible/
```

Sample response:


```
{"data":[{"name":"bonjovi","alias":"aI","target":"https://www.youtube.com/watch
    ?v=dPLfRZkJdbE","is_dangerous":false,"virus_info":"OK","is_public":true},{"
    name":"curl","alias":"Pz","target":"https://curl.se/","is_dangerous":false
    ,"virus_info":"OK","is_public":true},{"name":"google","alias":"-K","target
    ":"https://www.google.com/","is_dangerous":false,"virus_info":"OK","
    is_public":false},{"name":"wanderer","alias":"qV","target":"https://www.
    youtube.com/watch?v=f730CwvGUf8","is_dangerous":false,"virus_info":"OK","
    is_public":false}],"err":null}
```


6.2.4 Shorten a URL into a Link

Sample request:

```
curl -H 'Content-Type: application/json' \
     -H 'Authorization: Token 588d087c-58d1-4129-b610-d13a70309cee' \
     -d '{"name": "stackexchange", "url": "https://tex.stackexchange.com/"}'\
     -X POST \
     https://miniproject.gscoursework.thealois.com/api/links/new/
```

Sample response:


```
{"data":{"alias":"pV","created":true},"err":null}
```


6.2.5 Get Link Detail

Sample request:

```
curl -H 'Content-Type: application/json' \
     -H 'Authorization: Token 588d087c-58d1-4129-b610-d13a70309cee' \
     -X GET \
     https://miniproject.gscoursework.thealois.com/api/link/pv/
```

Sample response:

```
{"data":{"name":"stackexchange","alias":"pV","target":"https://tex.
    stackexchange.com/","is_dangerous":false,"virus_info":"OK","is_public":
    false},"err":null}
```

6.2.6 Delete a Link

Sample request:

```
curl -H 'Content-Type: application/json' \
     -H 'Authorization: Token 588d087c-58d1-4129-b610-d13a70309cee' \
     -X DELETE \
     https://miniproject.gscoursework.thealois.com/api/link/pv/
```

Sample response:

```
{"data":{"deleted":true,"link":{"name":"stackexchange","alias":"pV","target":"
    https://tex.stackexchange.com/","is_dangerous":false,"virus_info":"OK","
    is_public":false}},"err":null}
```

6.2.7 Make a Link Public

Sample request:

```
curl -H 'Content-Type: application/json' \
     -H 'Authorization: Token 588d087c-58d1-4129-b610-d13a70309cee' \
     -X POST \
     https://miniproject.gscoursework.thealois.com/api/link/make-public/-K/
```

Sample response:

```
{"data":"Made -K public.","err":null}
```

6.2.8 Make a Link Private

Sample request:

```
curl -H 'Content-Type: application/json' \
     -H 'Authorization: Token 588d087c-58d1-4129-b610-d13a70309cee' \
     -X POST \
     https://miniproject.gscoursework.thealois.com/api/link/make-private/-K/
```

Sample response:

```
{"data":"Made -K private.","err":null}
```

6.2.9 Ping

Sample request:

```
curl -H 'Content-Type: application/json' \
     -H 'Authorization: Token 588d087c-58d1-4129-b610-d13a70309cee' \
     -X GET \
     https://miniproject.gscoursework.thealois.com/api/ping/
```

Sample response:

```
{"data":{"message":"pong","is_authenticated":true,"whoami":"Linky Web Service
    "},"err":null}
```

Chapter 7

Advanced Techniques

7.1 Containerization

I use a Docker Compose file to make deployment extremely easy, while not needing to worry about toolchain versions.

7.2 Background Tasks

The virus check results from VirusTotal[1] are not instant, sometimes it could take several hours. So I implemented background tasks to wait for completion using bee-queue[6] and Redis[2].

7.3 Reverse Proxy

I use Caddy[4] for reverse proxy. It made certain steps a lot easier, such as SSL certificate automation, rate-limiting and region limiting.

7.4 Markdown Documentation Module from Scratch

The hugo[9] documentation theme was too heavy and there were not many options. So I created my own sub-site for documentation from scratch using several Node.js packages, and it's available at:

https://miniproject.gscoursework.thealois.com/docs/

7.5 LaTeX

This whole documentation PDF is compiled from LaTeX, which is reproducible and privacy-oriented. The source code for this documentation is in my code repository's `submission` directory.

7.6 Shell Scripting

I made some short but universal and useful shell scripts along the way, besides `startapp.sh`. For example `builddocs.sh` from previous commits, builds the documentation sub-site using dockerized hugo:

```
#!/usr/bin/env sh
HUGO_USER=${SUDO_USER:-${USER}}
docker container run --rm \
  -v ${PWD}/docs:/src \
  --entrypoint "sh"\
  hugomods/hugo:exts-0.120.4 \
  -c "hugo --minify && chmod -R 777 public && chown -R `id -u ${HUGO_USER}`
      public"

[ "$?" = "0" ] && rm -rf public/docs && mv docs/public public/docs
```

References

[1] Hispasec Sistemas. VirusTotal API: Cybersecurity Website, 2004 -. URL https://docs.virustotal.com/reference/overview. Accessed on 2023-12-10.

[2] Redis Ltd. Redis: The open source, in-memory data store used by millions of developers as a database, cache, streaming engine, and message broker., 2009 -. URL https://redis.io/. Accessed on 2023-12-10.

[3] Docker, Inc. Docker: Accelerated Container Application Development, 2013 -. URL https://www.docker.com/. Accessed on 2023-12-10.

[4] ZeroSSL. Caddy Server: a powerful, enterprise-ready, open source web server with automatic HTTPS written in Go, 2015 -. URL https://caddyserver.com/. Accessed on 2023-12-10.

[5] Hetzner Online GmbH. Hetzner: German Dedicated Server, Cloud, Storage & Hosting Provider, 1997 -. URL https://www.hetzner.com/. Accessed on 2023-12-10.

[6] Eli Skeggs and GitHub collaborators. bee-queue: A simple, fast, robust job/task queue for Node.js, backed by Redis, 2014 -. URL https://github.com/bee-queue/bee-queue. Accessed on 2023-12-10.

[7] Cyril Jaquier and GitHub collaborators. fail2ban: Daemon to ban hosts that cause multiple authentication errors., 2004 -. URL https://github.com/fail2ban/fail2ban. Accessed on 2023-12-10.

[8] Daniel Stenberg et al. curl: command line tool and library for transferring data with URLs , 1998 -. URL https://curl.se/. Accessed on 2023-12-10.

[9] GitHub contributers. hugo: The world's fastest framework for building websites., 2013 -. URL https://gohugo.io/. Accessed on 2023-12-10.